News Topic Classification from Text and Metadata: A Multiclass Classification Problem

Giovanni Indiano
Politecnico di Torino
Student ID: s357942

Fabio Stradiotti
Politecnico di Torino
Student ID: s359415

***Abstract*—This paper presents a robust machine learning pipeline for the multiclass classification of news articles. The proposed approach tackles the challenges of high dimensionality and class imbalance through a comprehensive preprocessing strategy, which includes custom text normalization, frequency-based categorical encoding, and cyclical feature transformation for temporal metadata. We engineer a high-dimensional feature space using TF-IDF vectorization and compare the performance of linear classification models. The experimental results identify the SVM as the optimal model, which outperforms a naive baseline defined for the problem.**

I. Problem overview

The proposed competition is a classification problem on a collection of approximately 100,000 instances describing online news articles collected from multiple international news sources. The goal of this competition is to develop and evaluate a machine learning model capable of automatically classifying news articles into 7 predefined categories (in order: International News, Business, Technology, Entertainment, Sports, General News, and Health) based on their textual content and associated metadata. The dataset is divided into two parts:

- a development set, containing 79,997 labeled instances
- an evaluation set of 20,000 unlabelled records.

The development set is used to build a classification model to correctly label the points in the evaluation set. Several considerations can be drawn from the development set. First, as Figure 1 shows, the problem is unbalanced. However, the adoption of Macro-F1 as the primary performance metric prevents the model from being biased toward more frequent classes, since the metric treats all classes equally regardless of their support size.

An initial exploratory analysis of the development set revealed formatting inconsistencies and data quality issues. Specifically, we observed non-standard null indicators such as the string literal "`\N`" and the double white spaces in textual fields and placeholder timestamps (0000-00-00 00:00:00) in the metadata.

Furthermore, the dataset contains duplicate entries which, if left unaddressed, could lead to data leakage or biased evaluation results.

II. Proposed approach

A. Preprocessing

Regarding the textual features, it is known that raw web-scraped text typically contains significant noise, including HTML markup, punctuation artifacts, and irrelevant technical metadata. To transform the unstructured text into a clean format suitable for feature extraction, we implement a custom preprocessing pipeline. The strategy is designed not just to remove noise, but to preserve latent information often lost during standard tag stripping. The preprocessing steps applied to both the `title` and `article` fields are as follows:

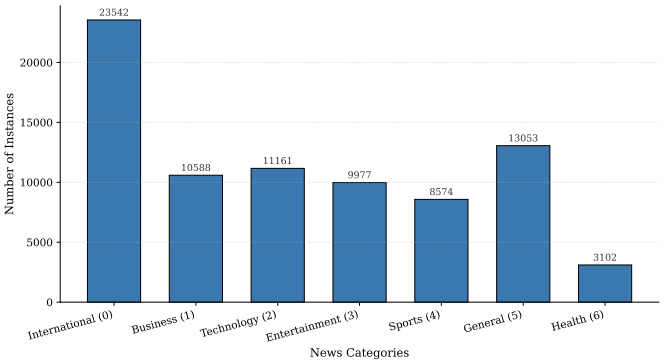


Fig. 1. Distribution of the news categories

1) We convert all text to lowercase to reduce vocabulary size and ensure case-insensitivity and we decode HTML entities (e.g., converting `&` to `&`).
2) Instead of discarding HTML tags and URLs entirely, we replace specific syntactic characters (e.g., `<`, `>`, `/`, `=`, `_`) with whitespace. This approach treats technical delimiters as word boundaries, allowing the recovery of meaningful tokens embedded within URLs or attributes (e.g., extracting "politics" from a URL string like `.../politics/2026...`).
3) We replace numerical digits with a generic token `NUM`, allowing the model to recognize the presence of quantitative information (common in categories like *Business* or *Sport*) without overfitting on specific scalar values (e.g., specific dates or prices).
4) We explicitly remove technical non-informative terms (e.g., `http`, `href`, `jpg`) and isolate single characters, which usually represent typographic residuals or noise.

Finally, we concatenate the preprocessed title and article into a single field, `text_content`, which serves as the input for the vectorization phase.

Prior to feature extraction, we implement a data cleaning pipeline to address the inconsistencies identified in the problem overview. We map the non-standard literal "\N" and the invalid timestamps to missing values (NaN) during the loading stage, ensuring that subsequent cleaning steps correctly identify invalid entries.

Subsequently, we drop rows containing missing values in both critical fields (article and title). To mitigate the impact of duplicated instances on model training, we implement a multi-step deduplication strategy. Initially, we target instances with identical `text_content`, defined as the concatenation of the title and the article body. For each group of identical entries, we apply a majority voting scheme:

- If a dominant class emerges, all instances are collapsed into a single representative entry with the majority label.
- In the event of a tie, all involved instances are discarded. This conservative approach prevents the model from learning from ambiguous or contradictory information.

A second refinement pass is performed by grouping instances based solely on the `article` field. This addresses cases where the same news body is paired with minor variations in the title. We apply the same majority voting and tie-breaking logic to these groups.

As a result of this process, the dataset size is reduced from 79997 to 74530 instances. This cleaning phase is crucial to ensure that the evaluation of the model is conducted on a reliable and non-redundant distribution of the data.

Following text normalization, we adopt the *Term Frequency-Inverse Document Frequency* (TF-IDF) weighting scheme to transform the unstructured textual data into numerical vectors. As demonstrated in the foundational study by Salton and Buckley [1], this approach offers a statistically significant advantage over simple count-based methods (e.g., Bag-of-Words). By penalizing ubiquitous tokens with low discriminative power and boosting rare, document-specific terms, TF-IDF enhances the separability of the classes in the vector space. A critical hyperparameter in text classification is the size of the sliding window for token extraction (N-grams). To capture local context and multi-word expressions common in news reporting (e.g., "European Union", "stock market crash"), we extend the vectorization to include unigrams, bigrams, and trigrams (range: $[1, 3]$). While extending to 4-grams theoretically captures longer dependencies, our experiments show that it leads to an exponential explosion in feature space dimensionality with negligible improvement in Macro-F1 score (< 0.001).

Beyond textual content, we analyze the available metadata. As shown in Figure 2, specific `PageRank` values are strongly associated with specific categories. Treating this feature as a continuous numerical variable would be suboptimal, as it implicitly assumes a monotonic relationship between the feature values and the target. Therefore, we treat `PageRank` as a categorical variable and we encoded it using One-Hot Encoding.

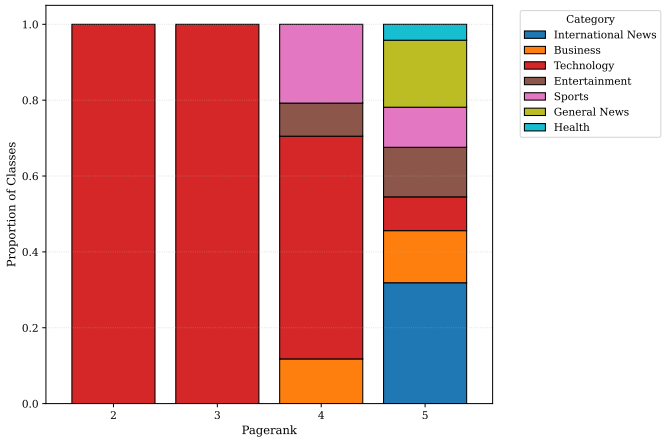


Fig. 2. Class Distribution by `PageRank` Value

Regarding the `timestamp` feature, there is a significant portion of NaN values across the dataset. To exploit this, we generate a binary feature `timestamp_missing` (1 if missing, 0 otherwise). Moreover, news topics often follow weekly cycles (e.g., sports events on weekends, financial reports on weekdays). We introduce a high-level boolean feature `is_weekend`, set to 1 for Saturday and Sunday, 0 otherwise. This explicit flag helps the model to easily split nodes based on the "weekday vs. weekend" dichotomy.

Moreover, simply encoding the hour as an integer $hour \in [0, 23]$ creates an ordinal discontinuity where midnight (0) appears mathematically distant from 11 p.m. (23). To preserve the temporal periodicity, we transform the hour into two continuous dimensions using sine and cosine functions [2]:

$$hour_{sin} = \sin\left(\frac{2\pi \cdot hour}{24}\right), \quad hour_{cos} = \cos\left(\frac{2\pi \cdot hour}{24}\right) \tag{1}$$

The dataset includes a `source` feature with 1340 unique categorical values, representing the news agencies. To prevent an excessive increase in the dimensionality of the feature space and avoid the curse of dimensionality, we adopt a frequency-based truncation strategy. The strategy consists of retaining only the top $N_{sources}$ most frequent sources as distinct categories. We aggregate all remaining, less frequent sources into a single catch-all category. The cutoff value $N_{sources}$ is treated as a structural hyperparameter. Following the sensitivity analysis detailed in Section II-C, we determine the optimal $N_{sources}$ and subsequently apply One-Hot Encoding to the resulting set of $N_{sources} + 1$ categories.

B. Model selection

The selection of classification models is driven by the high-dimensional and sparse nature of the TF-IDF feature space. We compare two distinct approaches:

- *SVM:* this algorithm aims to identify the maximum-margin hyperplane that separates the classes in the feature space. We employ a linear kernel, as the non-linear

kernels we evaluated were computationally prohibitive for our dataset. Moreover, linear SVMs have been shown to be particularly robust and effective in high-dimensional text categorization tasks [3].

- *Logistic Regression:* this model has also been shown to perform effectively in text classification tasks [4]. It is a fundamental statistical method that estimates class membership probabilities using a logistic function. While inherently a linear classifier, it provides a highly interpretable probabilistic framework.

To account for the inherent class imbalance within the dataset, we configure both models with the `class_weight='balanced'` parameter. This approach adjusts the cost function by assigning higher weights to minority classes, ensuring that the classifiers do not become biased toward the majority class. For both classifiers, we identify the optimal hyperparameter configuration through a grid search.

C. Hyperparameters tuning

There are two main sets of hyperparameters to be tuned:

- $N_{sources}$, `min_df` and `max_df` for the preprocessing
- Logistic Regression and SVM parameters

Specifically, the tuning includes both `min_df` and `max_df` to, respectively prune extremely rare tokens (often typos or noise) and reduce sparsity and filter out corpus-specific 3-grams that appear too frequently across documents to be discriminative. To reduce the computational complexity of the search space, we operate under the simplifying assumption that the choice of $N_{sources}$ is largely independent of the model's fine-tuning. Indeed, we can first select $N_{sources}$ and then move on to the classifiers parameters. We can use an 80/20 train/test split on the development set and run a grid search on $N_{sources}$. To this end, we train a Linear SVM and a Logistic Regression with their default configurations and assess their performance based on the resulting macro f1 score. After choosing a value for $N_{sources}$, we can run a grid search on both SVM and Logistic Regression, based on the hyperparameters defined in Table I.

Given the significant class imbalance observed during the exploratory data analysis, we employ a Stratified K-Fold cross-validation scheme ($K = 5$). This approach ensures that the class distribution is preserved across all folds, providing a more reliable and low-variance estimate of the Macro-F1 score. This rigorous validation framework is crucial to ensure that the selected hyperparameters generalize effectively to unseen data, mitigating the risk of a classification bias toward the majority classes, which is particularly relevant given the class imbalance observed in the development set.

Model	Parameter	Values

Preprocessing	$N_{sources}$
	`min_df`
	`max_df`
		$50 \longrightarrow 1250$, step 100
		$\{2, 3, 4, 5, 10\}$
		$\{0.4, 0.5, 0.75, 0.9\}$

SVM	`C`	$\{0.01, 0.1, 0.2, 0.3, 0.5, 1, 10\}$
Logistic Regression	`C` `solver`	$\{0.1, 1, 3, 5, 10\}$ {liblinear, saga}

TABLE I
HYPERPARAMETERS CONSIDERED

III. RESULTS

The tuning of the parameter $N_{sources}$ is summarized in Figure 3.

We can immediately observe that the performance rises sharply for low values ($N_{sources} < 50$) and then plateaus. Both classifiers achieve satisfactory performance for $N_{sources}$=150, which we can select as a reasonable value (larger values would increase the computational cost with no meaningful improvement in the F1-score). Fixing $N_{sources}$=150, we run the grid search for both Logistic Regression and Linear SVM. The best configuration for Logistic Regression is found for {C=3, max_df=0.5, min_df=2, solver=saga} achieving a Macro F1 score of 0.7293, whereas the best configuration for the SVM is found for {C=0.3, max_df=0.5, min_df=2}, achieving a Macro F1 score of 0.7334).

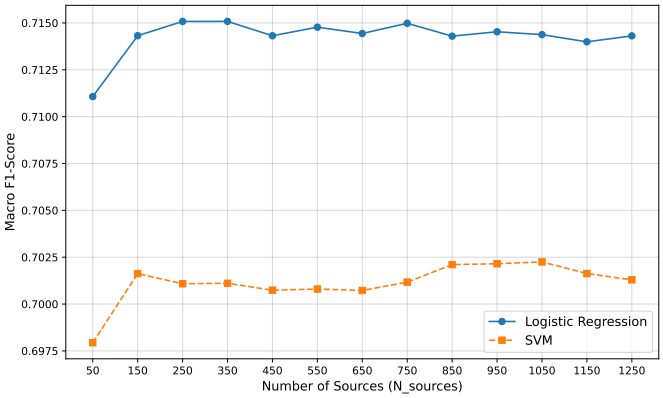


Fig. 3. Performance of default SVM and LogisticRegression as `N_sources` varies

We train the best performing SVM and Logistic Regression on all available development data. Then, we use the models to classify the instances in the evaluation set. The performance of both classifiers is summarized in Table II.

Although the two classifiers achieve comparable aggregate results, we select the SVM as our final model due to its slightly superior performance and robustness.

Model	Validation Set (Stratified CV)	Evaluation Set (Public Score)
Logistic Regression	0.7293	0.727
SVM	**0.7334**	**0.733**

TABLE II
COMPARISON OF MODEL PERFORMANCE

To better understand the model's limitations, we perform a detailed error analysis using the Confusion Matrix on the validation set (Figure 4).

The matrix reveals a significant variance in performance

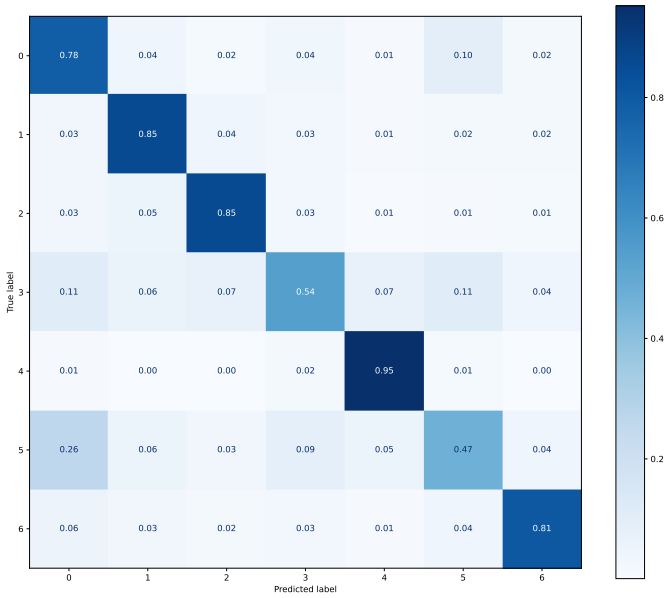


Fig. 4. Normalized Confusion Matrix for the SVM model

across different categories. While Classes 0 (International News), 1 (Business), 2 (Technology), and 4 (Sports) exhibit high recall rates (0.78, 0.85, 0.85, and 0.95 respectively), the model struggles significantly with Classes 3 (Entertainment) and 5 (General News). Specifically, Class 5 shows the lowest recall (0.47), with a substantial misclassification rate (26%) towards Class 0. Similarly, Class 3 (Recall 0.54) is frequently confused with Class 0 and Class 5.

These patterns suggest a semantic overlap between these specific categories in the TF-IDF feature space, which the linear decision boundary fails to separate effectively. However, it is important to note that these misclassifications are often logical. The distinction between these categories is inherently blurred: for instance, a news article reporting on price variations or economic shifts could plausibly be labeled as 'Business', 'International', or 'General' depending on the editorial angle. The model's confusion likely reflects this real-world ambiguity rather than a failure to learn distinct patterns.

Finally, it is worth noting that these results represent our initial submission on the evaluation data. The absence of a performance drop compared to our local validation suggests that the model has not overfitted the training set. Given this stability and the robustness demonstrated by the SVM, we can reasonably assume that similar performance levels will be maintained on the private leaderboard.

For comparison, a naive baseline has also been defined. This solution carries out the following steps:

- Concatenate the `title` and `article` fields into a single textual corpus.
- Transform textual data in numerical vectors using a standard TF-IDF vectorizer.
- Train a default Multinomial Naive Bayes model.

This configuration yields a public score of 0.458.

IV. Discussion

The results obtained demonstrate that the proposed classification pipeline performs effectively for the task of news article categorization. Both Logistic Regression and Linear Support Vector Machines achieved satisfactory and comparable results in terms of Macro-F1 score. A key strength of the selected approach is its computational efficiency. By opting for linear classification models instead of complex non-linear classifiers, the pipeline maintained a very low execution time of approximately 20 seconds. Despite the positive outcomes, several areas for improvement could be explored in future works:

- Other feature extraction approaches may be considered. Indeed, while the current n-gram approach captures local context effectively, it does not fully account for deep semantic relationships or long-range dependencies within the text. Future iterations could involve the integration of more advanced architectures, such as Deep Neural Networks (CNNs or RNNs) or pre-trained Transformer-based models like BERT. [5]
- To potentially enhance model performance, a more thorough *grid search* could be conducted. While the current study explores a specific subset of hyperparameters, expanding this search space may yield an even more optimal configuration. Furthermore, investigating a broader range of classification models could provide additional insights and further improve the results.

References

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information processing & management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] A. Zheng and A. Casari, *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*. O'Reilly Media, 2018.

[3] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," in *Proceedings of the 10th European Conference on Machine Learning (ECML)*, pp. 137–142, Springer, Apr. 1998.

[4] A. Genkin, D. Lewis, and D. Madigan, "Large-scale bayesian logistic regression for text categorization," *Technometrics*, vol. 49, 08 2007.

[5] M. Salih, S. Mohammed, A. Ibrahim, O. Ahmed, and L. Haji, "Fine-tuning bert for automated news classification," *Engineering, Technology Applied Science Research*, vol. 15, pp. 22953–22959, 06 2025.